Cloud Computing
Automation & Orchestration

Emiliano Casalicchio, Phd
Department of Computer Science
Sapienza University of Rome

Agenda

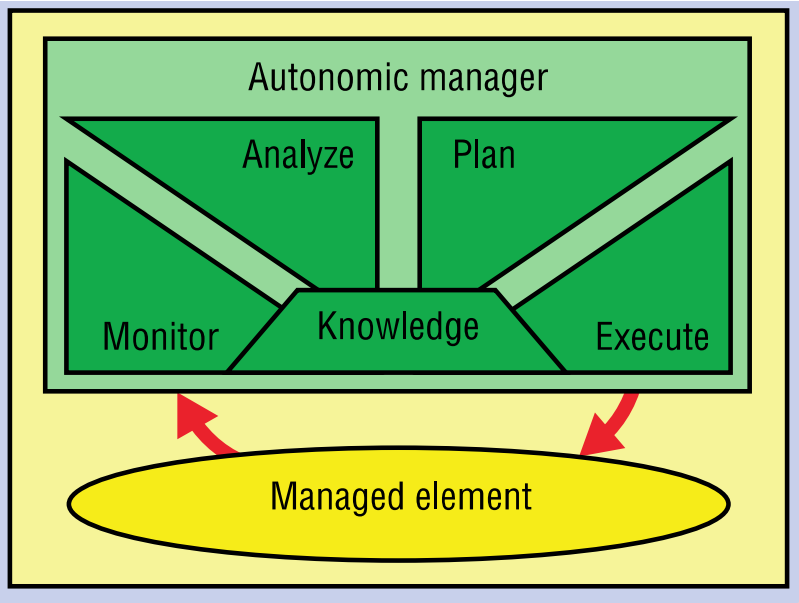
- recap

- Automation
  - Ansible as an example

- Orchestration
  - Kubernetes as an example

A recap

- What are the four self-* properties?



emiliano.casalicchio@uniroma1.it

IT Automation vs Autonomic computing

- Autonomic computing goal:
  - to keep the humans out of the loop (only high level objective are provided)
- Automation, in our case IT operations automation (ITOA) goal:
  - To reduces human interaction and manual processes

IT Operation Automation

- How
  - software tools and technologies to automate various organizational and operational IT processes.
- What
  - automating repetitive tasks, orchestrating workflows, and integrating automation into existing systems and applications.
- Benefits:
  - Improved efficiency and productivity: reduce the time and effort required to perform manual tasks. This frees up IT staff to focus on more strategic and value-added activities.
  - Reduced human error: eliminate the risk of mistakes caused by manual intervention. This can lead to more accurate and consistent results.
  - Saved time and costs: minimize the burden of repetitive and time-consuming tasks. Free up IT staff helps reduce the overall operational costs of the organization.

Some of the challenges in datacenters

- To consolidate servers (VM migration)
- To guarantee Service Level Agreements
- Manage hw/sw failures (discover, remediate, fix)
- Software updates
- Vulnerabilities/bugs fix

- **Which of them can be address with autonomic computing and which with automation?**

Some of the challenges in managing large scale virtual infrastructures or cloud application deployments

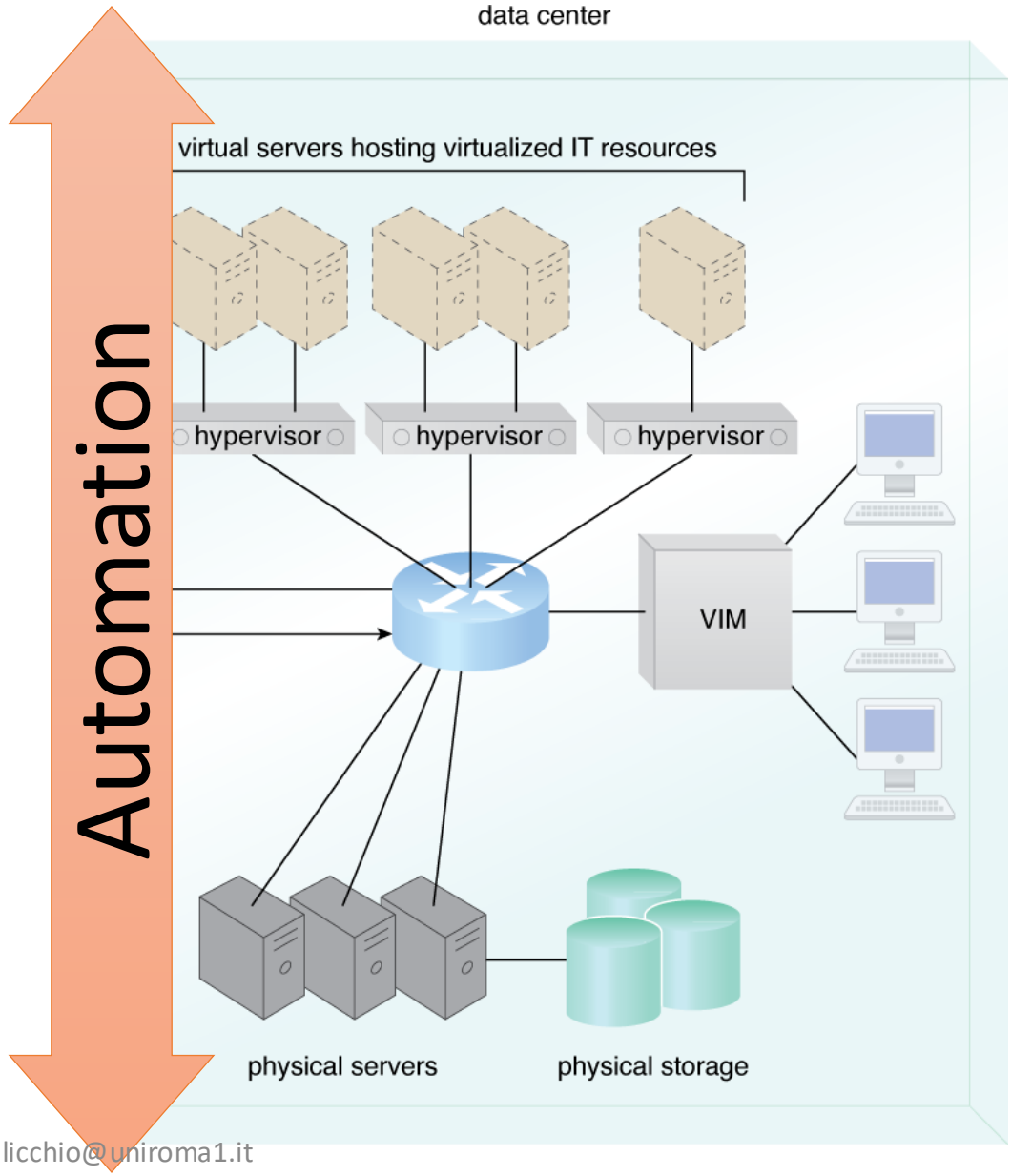
- Infrastructure nodes configuration
- Application components configuration
- Application components update and patching
  - Updates and patching need to be performed in the right order
- Backup
- CI/CD
- **Which of them can be address with autonomic computing and which with automation?**

Automation tools (some of them)

- Ansible
- Selenium
- Nintex
- Terraform
- Fortra
- Keap
- Pegasystems
- Tidal Software
- Jenkins
- Kissflow
- Vagrant
- Buildbot
- Kubernetes
- Splunk
- Jira
- Worksoft
- Chef
- Salt
- Docker
- Robotic process automat...
- Gradle
- TeamCity
- Endpoint Central
- Zapier

Automation

- Data centers have specialized platforms that automate tasks like
  - provisioning,
  - configuration,
  - patching, and
  - Monitoring
  - Continuous Integration and delivery (CI/CD)

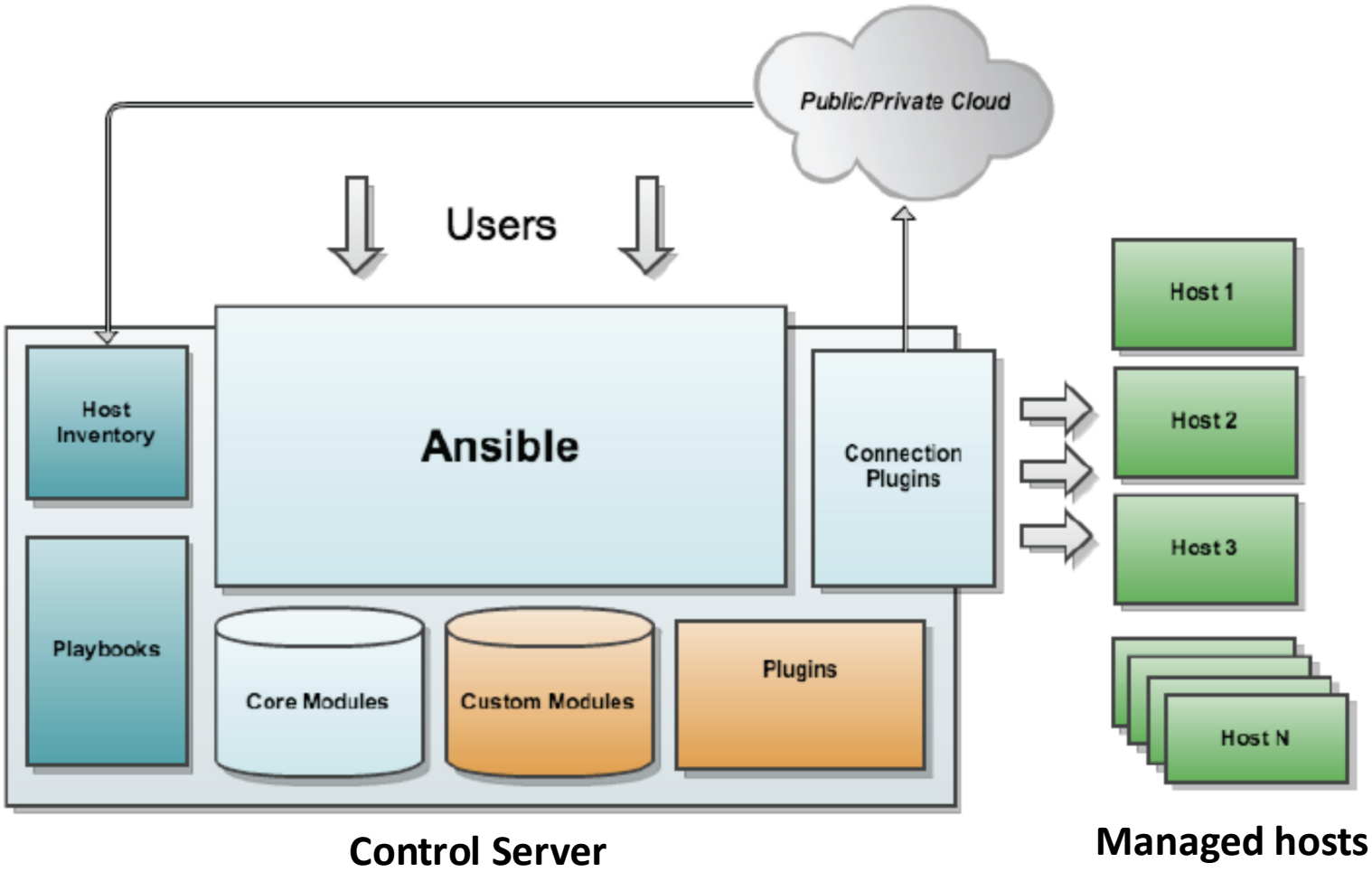


emiliano.casalicchio@uniroma1.it
Copyright © Arcitura Education

Ansible (to provide a practical example)

Reference: https://docs.ansible.com/ansible/latest/index.html

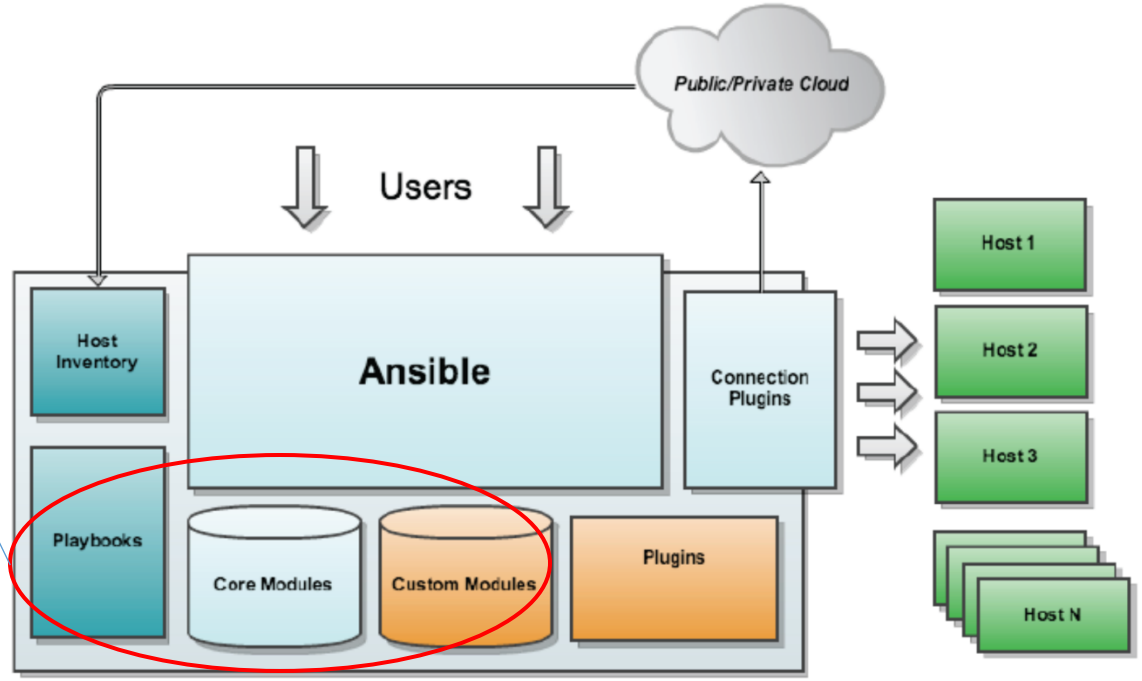
- Is an open source IT configuration management, deployment, and orchestration tool
- Is an **agentless** tool that runs in a **'push' model**
  - But can be reverted in pull mode



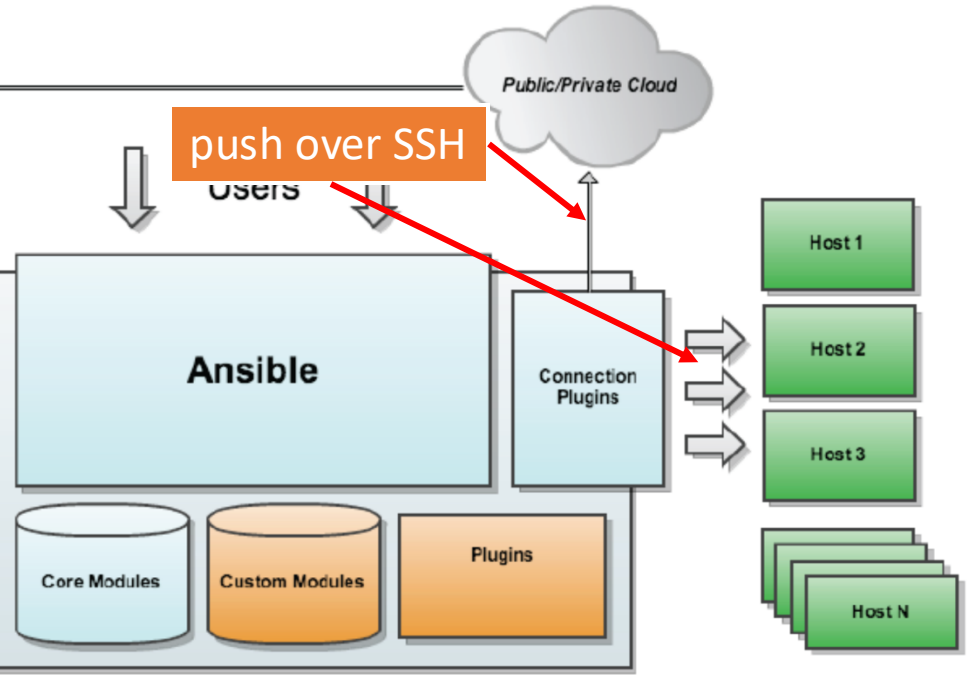
Agentless

- No software is required to be installed on remote machines to make them manageable
- No overhead when management is not running
- Ideal for high-security environments or high-performance cases where there are concerns about stability or permanence of a management agent

Needed control instructions are prepared and stored on the control server and then transferred (push) to the host to be managed

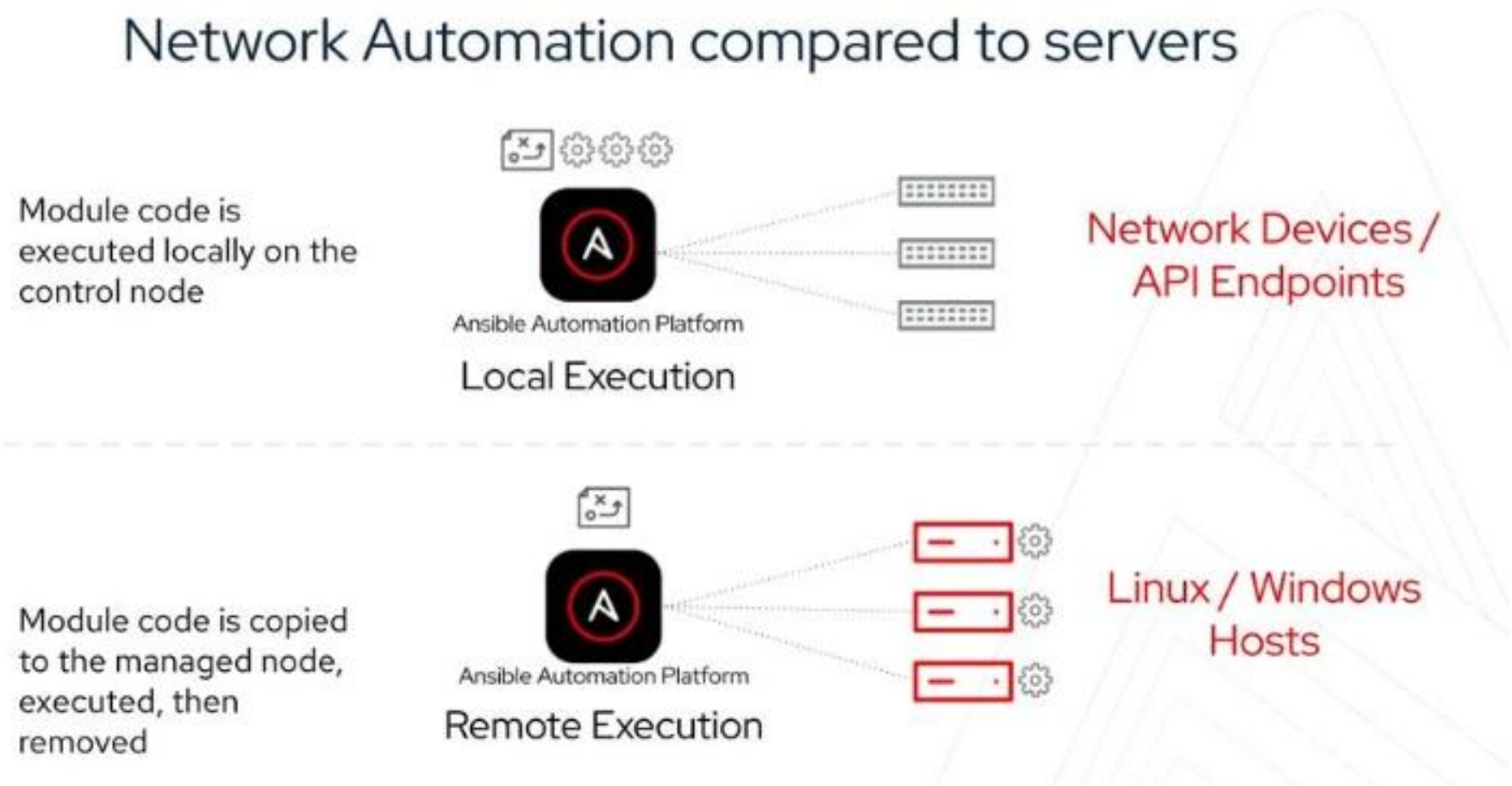


Push model



- Manages remote machines by using remote management framework already existing on the platforms
  - over SSH (Linux and UNIX)
  - over WinRM (Windows)
- Needed code (called Ansible 'modules') are passed to remote machines (push)
  - remote machines cannot see or affect how other machines are configured

Not all the nodes can run code (like network devices)



https://www.ansible.com/overview/how-ansible-works

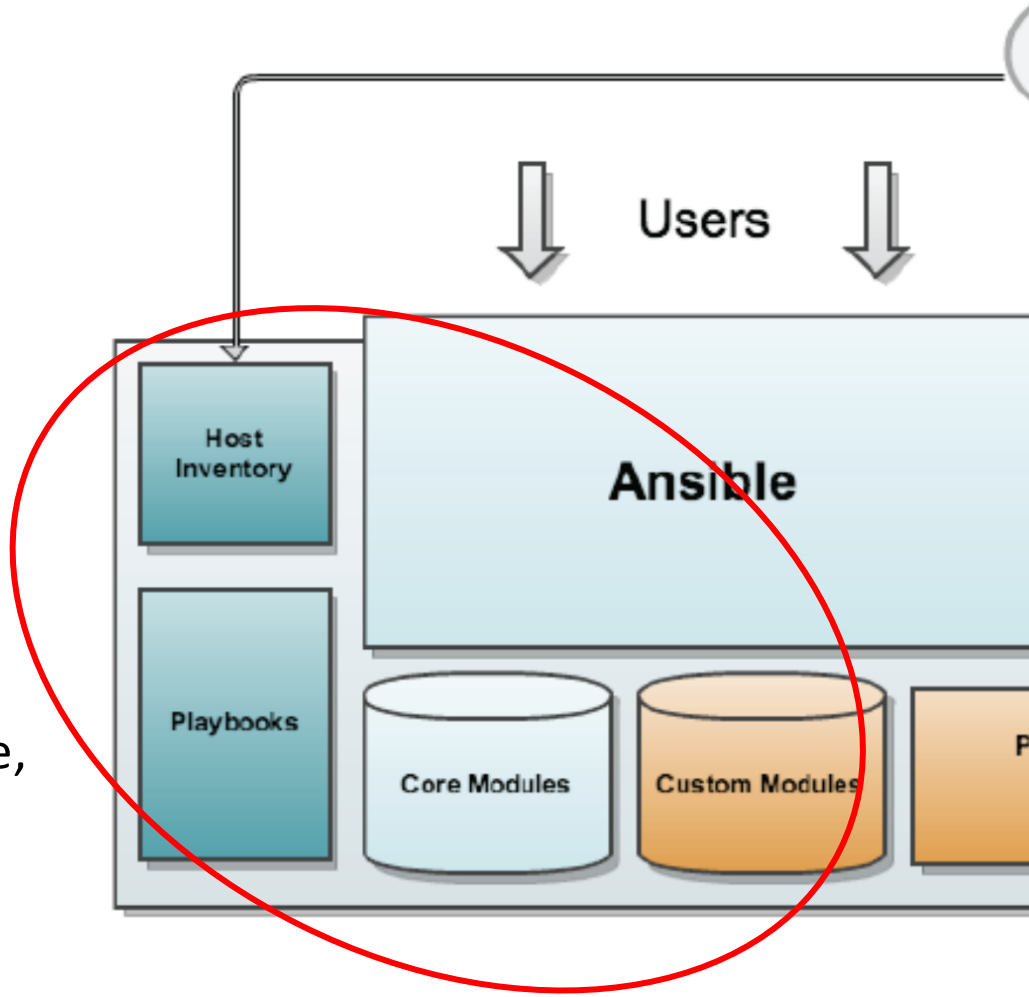
Playbooks

https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_intro.html

- Ansible Playbooks
  - Allow simple configuration management and multi-machine deployment system
  - Are repeatable and reusable
  - Use a YAML syntax
- Playbooks can:
  - declare configurations
  - orchestrate steps of any manual ordered process, on multiple sets of machines, in a defined order
  - launch tasks synchronously (default) or asynchronously (for long running task)
  - define the desired state
  - can be idempotent

Playbooks (cont'd)

- Consist of series of 'plays' that define automation across a set of hosts, known as the 'inventory'
- Each 'play' consists of multiple 'tasks,' that can target one, many, or all of the hosts in the inventory
- Each task is a call to an Ansible module - a small piece of code for doing a specific task
  - Simple - placing a configuration file on a target machine, or installing a software package.
  - Complex - spinning up an entire CloudFormation infrastructure in Amazon EC2



Core modules

- Written in an manner to allow for easy configuration of desired state
  - they check that the task that is specified actually needs to be done before executing it
- For example, if an Ansible task is defined to start a webserver, configuration is only done if the webserver is not already started.
  - This desired state configuration ensures that configuration can be applied repeatedly without side effects, and that configuration runs quickly and efficiently when it has already been applied

An example

```yaml
---
- name: Update web servers
  hosts: webservers
  remote_user: root

  tasks:
  - name: Ensure apache is at the latest version
    ansible.builtin.yum:
      name: httpd
      state: latest
  - name: Write the apache config file
    ansible.builtin.template:
      src: /srv/httpd.j2
      dest: /etc/httpd.conf

- name: Update db servers
  hosts: databases
  remote_user: root

  tasks:
  - name: Ensure postgresql is at the latest version
    ansible.builtin.yum:
      name: postgresql
      state: latest
  - name: Ensure that postgresql is started
    ansible.builtin.service:
      name: postgresql
      state: started
```

Play 1: target Web Server

Play 2: target DB Server

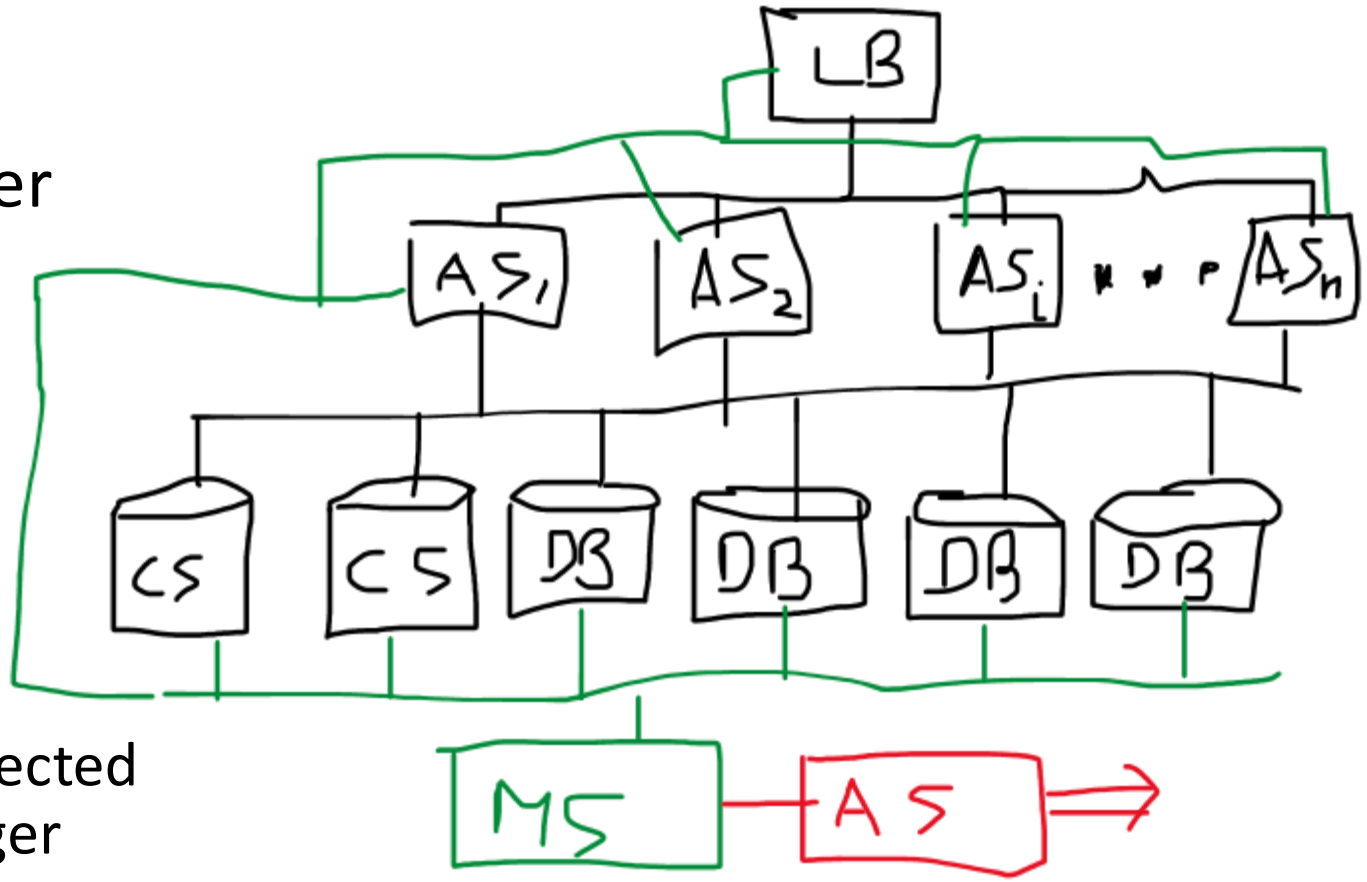
From https://docs.ansible.com/ansible/latest/user_guide/playbooks_intro.html

- ansible.builtin.yum - core module that check if the httpd is at the latest version. If not it is updated
- ansible.builtin.template - core module that write the file from src to dest
- ansible.builtin.service – core module that bring a service in the desired state

icchio@uniroma1.it

An example

- Consider a traditional three-tier web application and its environment consisting of:
  - Application servers
  - Database servers
  - Content servers
  - Load balancers
  - Plus, a monitoring system connected to an alert system such as a pager notification service



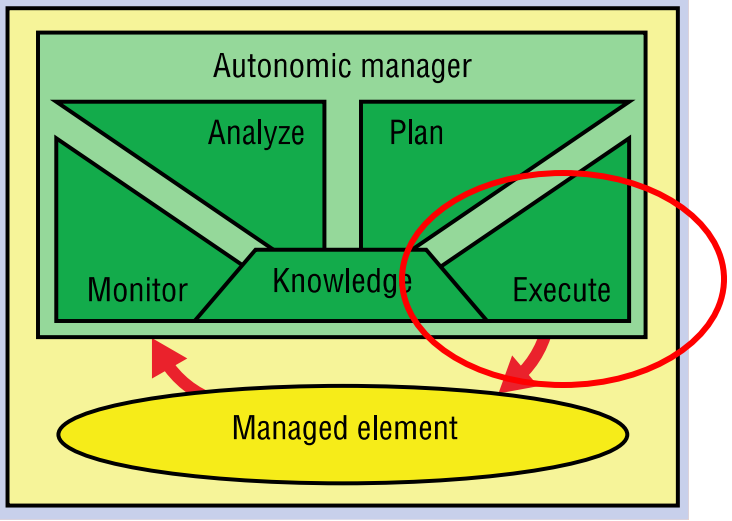
An example (cont'd)

- Ansible can easily be used to implement a complex, cluster-wide rolling update process that consists of:
  1. Consulting a configuration/settings repository for information about the involved servers
  2. Configuring the base OS on all machines and enforcing desired state
  3. Identifying a portion of the web application servers to update
  4. Signaling the monitoring system of an outage window prior to bringing the servers off-line
  5. Signaling load balancers to take the application servers out of a load balanced pool
  6. Stopping the web application server

An example (cont'd)

7. Deploying or updating the web application server code, data, and content
8. Starting the web application server
9. Running appropriate tests on the new server and code
10. Signaling the load balancers to put the application servers back into the load balanced pool
11. Signaling the monitoring system to resume alerts on any detected issues on those servers
12. Repeating this process for remaining application servers in a rolling update process
13. Repeating these rolling update processes for other tiers such as database or content tiers
14. Sending email reports and logging as desired when updates are complete

Automation and Autonomic computing



- Self-configuration
  - When decided the reconfiguration, a playbook can be used to deploy the new system configuration
- Self-optimization
  - When decided for adding new servers, a playbook doing all the required actions can be executed
- Self-healing
  - If a server/application performance are degrading, a playbook that detach and restart the server/application can be executed
- Self-protection
  - If a system is under attack because a vulnerability, a playbook for rolling an update can be deployed

Orchestration

emiliano.casalicchio@uniroma1.it

Orchestration vs Automation

- Automation
  - Setting up one task to run on its own

- Orchestration
  - Automating a series of individual tasks to work together as a process or workflow

Orchestration

- Orchestration concept could be specialized depending on the context
  - Service orchestration (we already presented this concept)
  - IT infrastructure orchestration
  - Container orchestration

Container Orchestration

- Allows cloud and application providers to define how to select, deploy, monitor, and dynamically control the configuration of multicontainer packaged applications in the cloud
- Is concerned with the management at runtime to support the Deploy, Run, and Maintain phases
- usually offer the following main features: resource limit control, scheduling, load balancing, health check, fault tolerance, and auto-scaling.

Concurrency and Computation
Practice and Experience

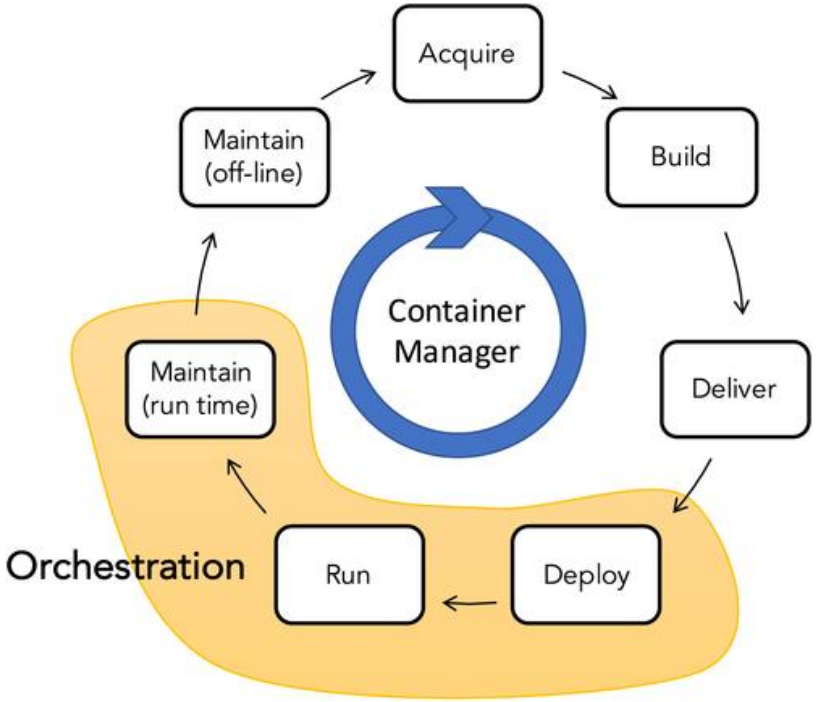
SPECIAL ISSUE PAPER | Full Access

The state-of-the-art in container technologies: Application, orchestration and security

Emiliano Casalicchio, Stefano Iannucci

First published: 19 January 2020 | https://doi.org/10.1002/cpe.5668 | Citations: 42

https://onlinelibrary.wiley.com/doi/full/10.1002/cpe.5668



The container lifecycle. A container manager provides API to support at least from the acquire to the run phases, while the Orchestrator allows to automatize the deployment, run, and maintain (run-time) phases

Orchestration features	Orchestration frameworks			
	Swarm	**Kubernetes**	**Cloudify**	**Marathon**
Resource limit control	CPU, Memory		N.A.	N.A.
Scheduling	Resource constrained, Node affinity, Custom			
Load balancing	Round-Robin, Custom		N.A.	Custom
Health check	Open port connections (UDP, TCP, SSL), HTTP req/res			
Fault tolerance	Replica control, High availability controller			N.A.
Auto-scaling	Custom	Custom, Threshold based (CPU)		Custom

Features implemented by different orchestration frameworks (2019/20)

Agenda

- Kubernetes
  - Architecture
  - Objects
  - Pods
  - Node
  - Control plane components
  - Node components
  - Scheduler

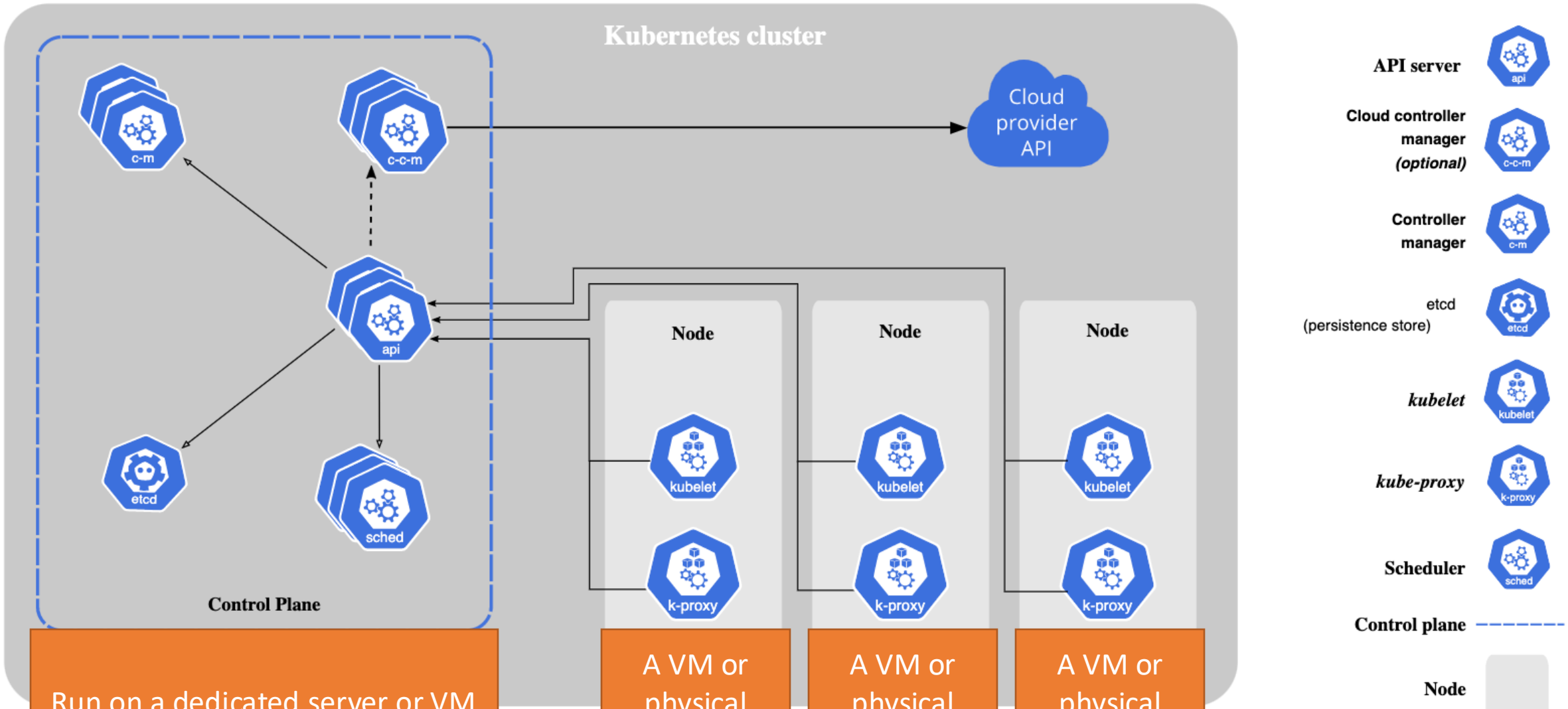
https://kubernetes.io/docs/home/

Kubernetes

- Kubernetes (K8s) is a platform for managing containerized workloads and services
- What does it meas to manage containerized workload?
  - **Service discovery and load balancing**
  - **Storage orchestration;** automatically mount storage
  - **Automated rollouts and rollbacks;** automatically bring the deployed containers in a desired (described) state
  - **Automatic bin packing;** fit containers onto cluster nodes to make best use of resources; for each container is give the needed CPU and memory
  - **Self-healing;** restart, replace, kill, don’t advertise faulty containers
  - **Secret and configuration management;**

K8s cluster

https://kubernetes.io/docs/concepts/overview/components/



Run on a dedicated server or VM

A VM or physical server

A VM or physical server

A VM or physical server

K8s cluster' components

https://kubernetes.io/docs/concepts/overview/components/

- Control Plane components
  - kube-apiserver
  - etcd
  - kube-scheduler
  - kube-controller-manager
  - cloud-controller-manager
- Node components
  - Kubelet
    - the primary "node agent" that runs on each node. It can register the node with the apiserver
  - kube-proxy
    - a network proxy that runs on each node in a cluster, implementing part of the Kubernetes Service concept
    - maintains network rules that allow network communication between the node and the cluster/internet
  - Container runtime

- We need three concepts
  - Objects
    - https://kubernetes.io/docs/concepts/overview/working-with-objects
  - Pods
    - https://kubernetes.io/docs/concepts/workloads/pods/
  - Nodes/Node status
    - https://kubernetes.io/docs/concepts/architecture/nodes/

Reference https://kubernetes.io/docs/concepts/overview/working-with-objects/

K8s Objects

- *K8s objects* are persistent entities in K8s and are used to represent the state of the cluster. Specifically, they can describe:
  - What containerized applications are running (and on which nodes)
  - The resources available to those applications
  - The policies around how those applications behave, such as restart policies, upgrades, and fault-tolerance
- A K8s object is a "record of intent"--once the object is created, the K8s system will constantly work to ensure that object exists.

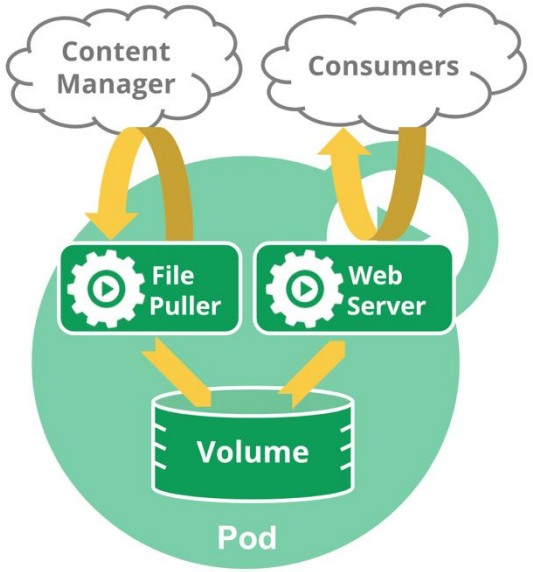
K8s Objects: spec & status

- An object contains two fields:
  - spec – describe the **desired state**
    - E.g. 3 instances of the application
  - status – describe the **current state** of the object
    - E.g. 2 instances are currently up&running

K8s Objects: spec & status

- Example (https://kubernetes.io/docs/concepts/overview/working-with-objects/kubernetes-objects/)
  - a Deployment is an object that can represent an application running on a cluster
  - You create the Deployment and set the Deployment **spec:**
    - **spec:** you want three replicas of the application to be running.
  - The Kubernetes system reads the Deployment spec
    - starts three instances of your desired application
    - updates the status to match your spec.
  - If any of those instances should fail (a status change)
    - the K8s responds to the difference between spec and status by starting a replacement instance

Pods



- The smallest deployable units of computing that can be created and managed in K8s.
- Models an application-specific "logical host": it contains one or more application containers which are relatively tightly coupled, and share
  - Storage resources
  - Network resources
  - A specification for how to run the containers.
- A Pod's contents are always co-located and co-scheduled, and run in a shared context.
- The shared context of a Pod is a set of Linux namespaces, cgroups
  - Within a Pod's context, the individual applications may have further sub-isolations applied.
- A Pod is similar to a set of containers with shared namespaces and shared filesystem volumes.

K8s Nodes

https://kubernetes.io/docs/concepts/architecture/nodes/

- A virtual or physical machine, member of a K8s cluster
- Each node is managed by the control plane and contains the services necessary to run Pods
- Nodes run containerized workload (Pods)
- Nodes became member of a K8s cluster in two way
  - The kubelet on a node self-registers to the control plane (API-server)
  - A human user manually adds a Node object to the API-server
- Once a Node object is created the control plane checks whether the new Node object is valid (all the service are running). If the node object is valid it could run a Pod

JSON manifest describing a node to be created

```
{
  "kind": "Node",
  "apiVersion": "v1",
  "metadata": {
    "name": "10.240.79.157",
    "labels": {
      "name": "my-first-k8s-node"
    }
  }
}
```


emiliano.casalicchio@uniroma1.it

K8s node status

- Described by
  - Address
    - Hostname, IP (external/internal)
  - Condition

Node Condition	Description
Ready	`True` if the node is healthy and ready to accept pods, `False` if the node is not healthy and is not accepting pods, and `Unknown` if the node controller has not heard from the node in the last `node-monitor-grace-period` (default is 40 seconds)
DiskPressure	`True` if pressure exists on the disk size—that is, if the disk capacity is low; otherwise `False`
MemoryPressure	`True` if pressure exists on the node memory—that is, if the node memory is low; otherwise `False`
PIDPressure	`True` if pressure exists on the processes—that is, if there are too many processes on the node; otherwise `False`
NetworkUnavailable	`True` if the network for the node is not correctly configured, otherwise `False`

JSON structure describing an healty node

```
"conditions": [
  {
    "type": "Ready",
    "status": "True",
    "reason": "KubeletReady",
    "message": "kubelet is posting ready status",
    "lastHeartbeatTime": "2019-06-05T18:38:35Z",
    "lastTransitionTime": "2019-06-05T11:41:27Z"
  }
]
```

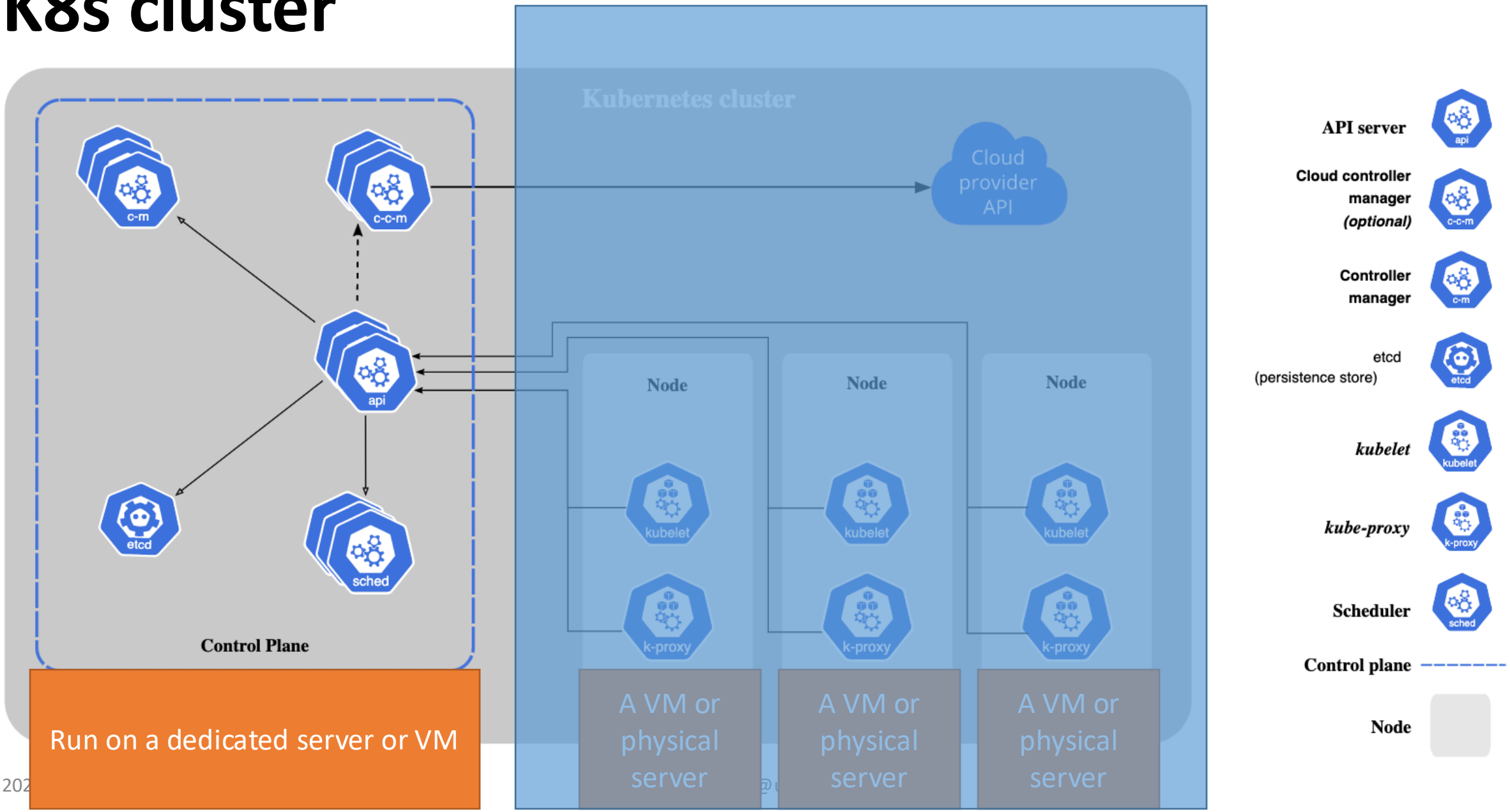
If status is unknown or false the node controller triggers API-initiated eviction for all Pods assigned to that node.
The default eviction timeout duration is 5 minutes

ano.casalicchio@uniroma1.it

K8s node status

- Described by (cont'd)
  - Capacity
    - indicates the resources available on the node: CPU, memory, and the maximum number of pods that can be scheduled onto the node.
  - Allocable
    - indicates the amount of resources on a Node that is available to be consumed by normal Pods
  - Info
    - general information about the node, such as kernel version, Kubernetes version (kubelet and kube-proxy version), container runtime details, and which operating system the node uses.

K8s cluster


Kubernetes cluster
Cloud provider API
c-m
c-c-m
api
etcd
sched
Control Plane
Node
Node
Node
kubelet
kubelet
kubelet
k-proxy
k-proxy
k-proxy
A VM or physical server
A VM or physical server
A VM or physical server
Run on a dedicated server or VM
API server
Cloud controller manager (optional)
Controller manager
etcd (persistence store)
kubelet
kube-proxy
Scheduler
Control plane
Node

K8s control plane components

- **kube-apiserver**
  - Front-end of the K8s control plane
  - Scale horizontally;
- **etcd**
  - Consistent and highly-available key value store used as Kubernetes' backing store for all cluster data
- **kube-scheduler**
  - watches for newly created **Pods** with no assigned node, and selects a node for them to run on
  - scheduling decisions based on: (https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/)
    - individual and collective resource requirements,
    - hardware/software/policy constraints,
    - affinity and anti-affinity specifications (https://overcast.blog/mastering-node-affinity-and-anti-affinity-in-kubernetes-db769af90f5c)
    - data locality,
    - inter-workload interference
    - deadlines

Node Affinity

- Node affinity in Kubernetes is a set of rules used to specify preferences that affect how pods are placed on nodes. It allows to constrain which nodes your pod is eligible to be scheduled on, based on labels on nodes and if those labels match the rules
- Useful for
  - Ensuring Compliance with Data Sovereignty Laws
  - Optimizing Network Latency for Distributed Systems
  - Allocating Resources for High Performance Computing (HPC)
  - Handling Workloads with Specific Storage Requirements
  - Supporting Multi-Tenancy and Resource Isolation

Node Anti-Affinity

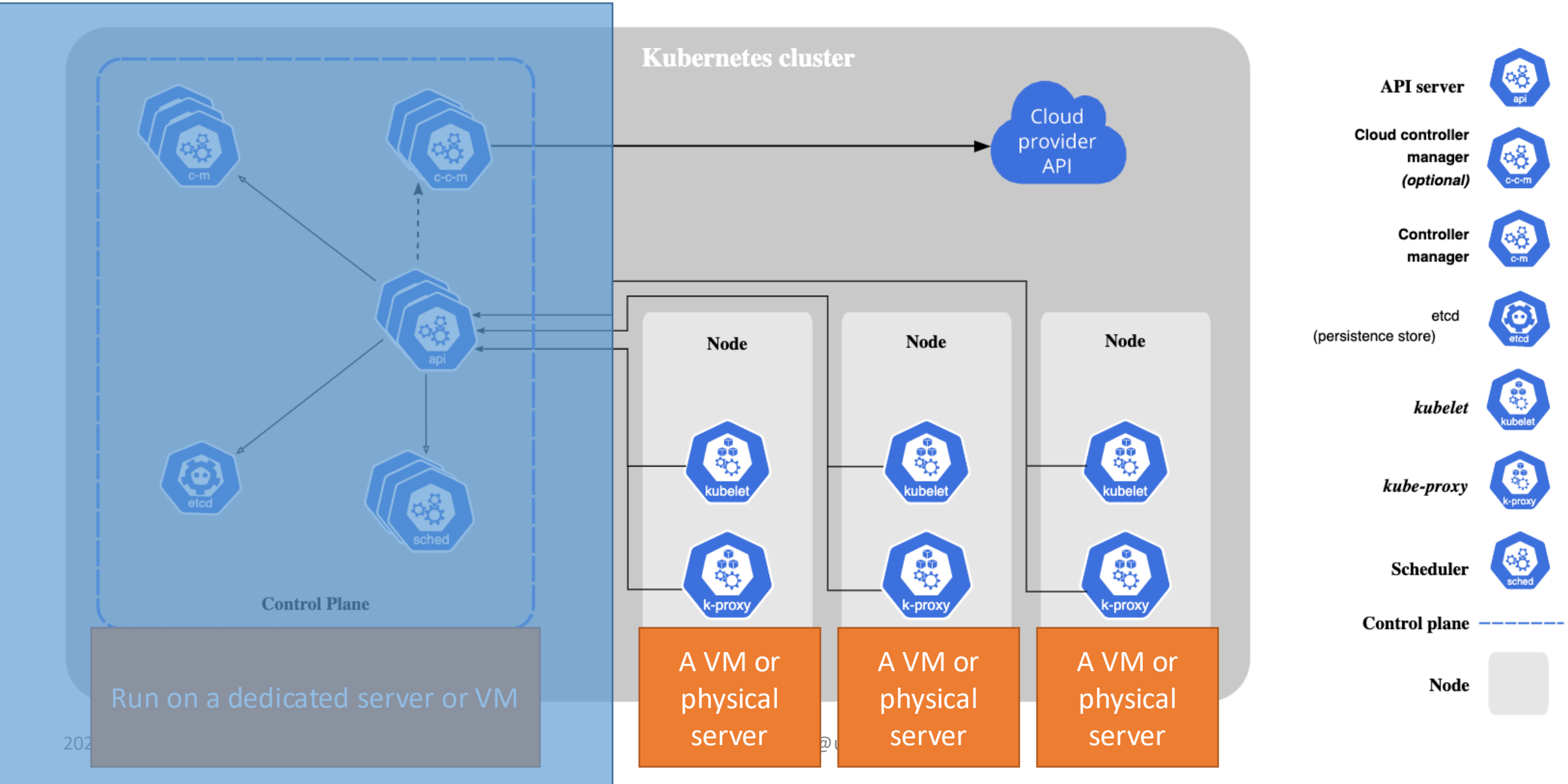
- Node anti-affinity is used to specify rules that prevent the scheduling of pods in the same or different nodes. It is particularly useful in
  - high-availability deployments where spreading pods across different nodes or zones can reduce the risk of a single point of failure
  - Separating Workloads for keeping certain pods from being co-located on the same node for performance or security reasons

K8s control plane components (cont'd)

- kube-controller-manager
  - Runs controller processes, e.g.
    - Node controller: Responsible for noticing and responding when nodes go down.
    - Job controller: Watches for Job objects that represent one-off tasks, then creates Pods to run those tasks to completion.
    - EndpointSlice controller: Populates EndpointSlice objects (to provide a link between Services and Pods).
    - ServiceAccount controller: Create default ServiceAccounts for new namespaces

- cloud-control-manager
  - To link a K8s cluster with a specific cloud provider's API

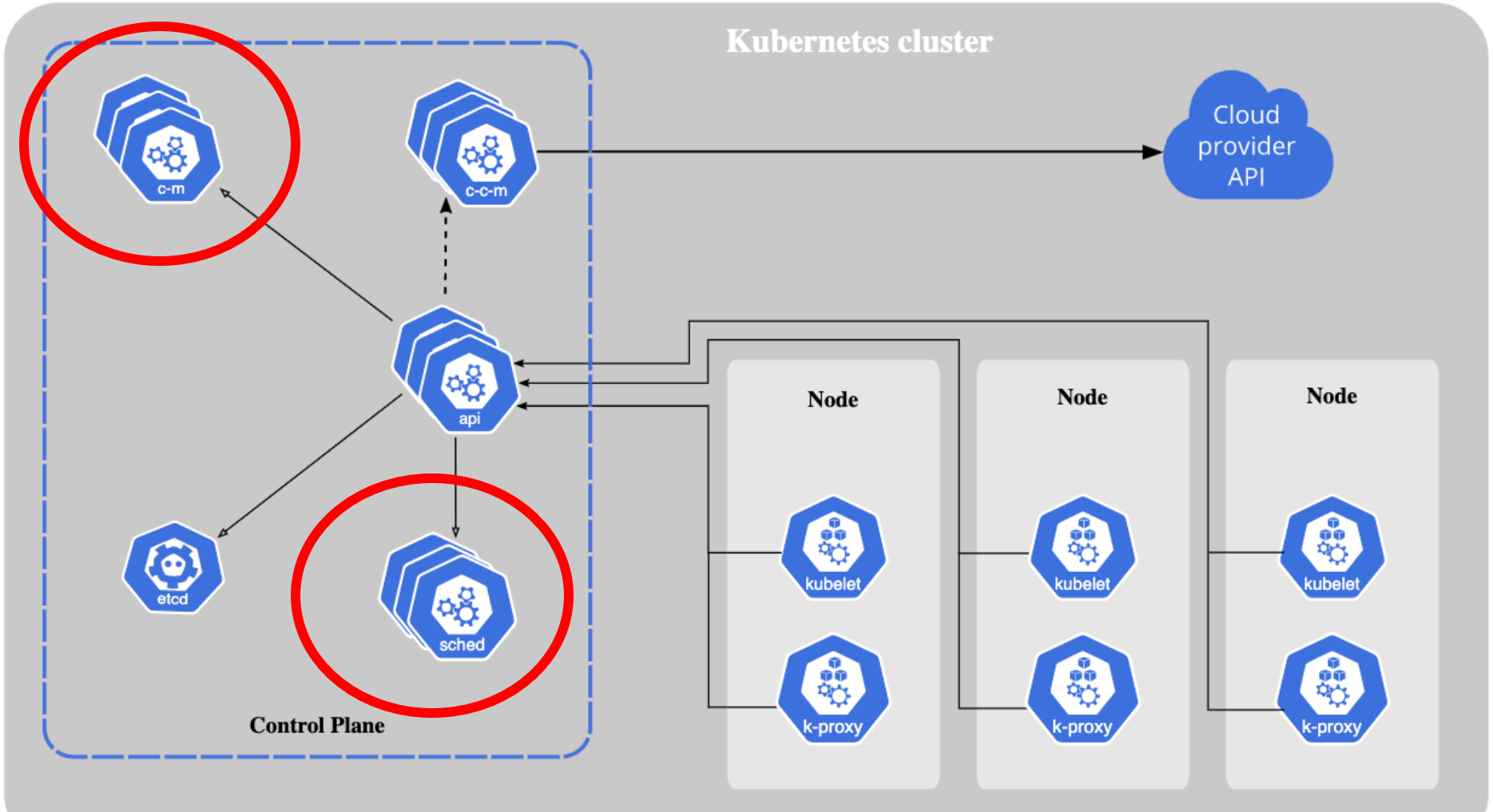
K8s cluster


Kubernetes cluster
Cloud provider API
c-m
c-c-m
api
etcd
sched
Control Plane
Node
Node
Node
kubelet
k-proxy
Run on a dedicated server or VM
A VM or physical server
A VM or physical server
A VM or physical server
API server
Cloud controller manager (optional)
Controller manager
etcd (persistence store)
kubelet
kube-proxy
Scheduler
Control plane
Node

K8s Node components

- kubelet
  - An agent running on each node
  - Control that containers described in a PodSpec are running and healty
  - PodSpecs: input for the kubelet
- kube-proxy
  - a network proxy that runs on each node
  - maintains network rules on nodes. These network rules allow network communication to cluster’s Pods from network sessions inside or outside of the cluster
- container runtime
  - the software that is responsible for running containers, e.g. Docker or CRI-O

- Now we jump back to the control plane



API serv

Cloud control manag (optior

Control manag

etc (persistence store)

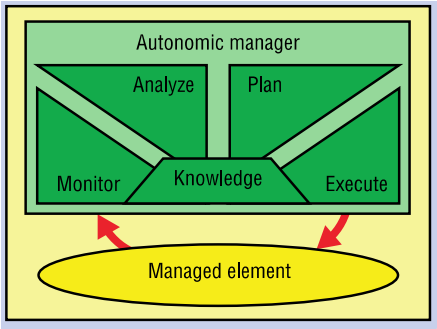
kube

kube-pro

Schedu

Control pla

K8s Controllers



- Controllers are control loops that watch the state of a K8s cluster, then make or request changes where needed.
- A controller tracks at least one Kubernetes resource type.
  - These objects have a spec field that represents the desired state.
  - The controller(s) for that resource are responsible for making the current state come closer to that desired state.
- A controller will send messages to the API server that have useful side effects

Example: Job controller

- Job is a Kubernetes resource that runs a Pod, or perhaps several Pods, to carry out a task and then stop
  - Once scheduled, Pod objects become part of the desired state for a kubelet
- When the Job controller sees a new job it makes sure that, somewhere in your cluster, the kubelets on a set of Nodes are running the right number of Pods to get the work done.
- The Job controller tells the API server to create or remove Pods.
  - Other components in the control plane act on the new information (there are new Pods to schedule and run), and eventually the work is done.
- After a new Job is created, the desired state is for that Job to be completed.
  - The Job controller makes the current state for that Job be nearer to your desired state: creating Pods that do the work you wanted for that Job, so that the Job is closer to completion.
- Once the work is done for a Job, the Job controller updates that Job object to mark it Finished

Example: Control of external cluster resources (Direct control)

- For example: **to make sure there are enough Nodes in your cluster**
  - K8s has a controller that horizontally scale the nodes in a cluster
    - Cluster autoscaler
    - https://github.com/kubernetes/autoscaler/
- Controllers that interact with external state find their desired state from the API server, then communicate directly with an external system to bring the current state closer in line.

Scheduler

- In K8s, scheduling refers to making sure that Pods are matched to Nodes so that Kubelet can run them
- A scheduler watches for newly created Pods that have no Node assigned.
  - For every Pod that the scheduler discovers, the scheduler becomes responsible for finding the best Node for that Pod to run on
- Kube-scheduler is the default scheduler;
  - it is possible to implement custom schedulers

Kube-scheduler

- For every newly created pod or unscheduled pods, selects an optimal node
  - Every container in pods has different requirements for resources
  - Pod also has different requirements for resources
  - Existing nodes need to be filtered according to the specific scheduling requirements
- Nodes that meet the scheduling requirements for a Pod are called feasible nodes
  - If none of the nodes are suitable, the pod remains unscheduled until the scheduler is able to place it.

Kube-scheduler (cont'd)

- The scheduler
  - finds feasible Nodes for a Pod (**Filtering**)
  - then runs a set of functions to score the feasible Nodes (**Scoring**)
  - picks a Node with the highest score among the feasible ones to run the Pod.
  - notifies the API server about this decision in a process called **binding**.
- Example of factors that need to be taken into account for scheduling decisions
  - individual and collective resource requirements,
  - hardware / software / policy constraints,
  - data locality,
  - inter-workload interference

Node selection in Kube-scheduler

- Filtering
  - finds the set of Nodes where it's feasible to schedule the Pod
    - PodFitsResources filter checks whether a candidate Node has enough available resource to meet a Pod's specific resource requests
    - Can explore all the cluster’s nodes or only a subset (e.g. 50% of the nodes)
  - Produces the node list containing any suitable Nodes;
    - often, there will be more than one.
    - If the list is empty, that Pod isn't (yet) schedulable
- Scoring
  - The scheduler assigns a score to each Node that survived filtering, basing this score on the active scoring rules.
  - Finally, kube-scheduler assigns the Pod to the Node with the highest ranking.
    - If there is more than one node with equal scores, kube-scheduler selects one of these at random.

Configuration of Filtering and Scoring

- Scheduling Policies (not supported since v1.23)
  - allow you to configure
    - Predicates for filtering
    - Priorities for scoring.
- Scheduling Profiles
  - allow you to configure Plugins that implement different scheduling stages, including:
    - QueueSort, Filter, Score, Bind, Reserve, Permit, and others
  - Used to customize kube-scheduler

Outcome

- General concept of Automation and Orchestration
- Difference between Autonomic computing and Automation
- Difference between Automation and Orchestration
- An example of how an automation tool works (Ansible)
- An example of how an orchestrator works (K8s)